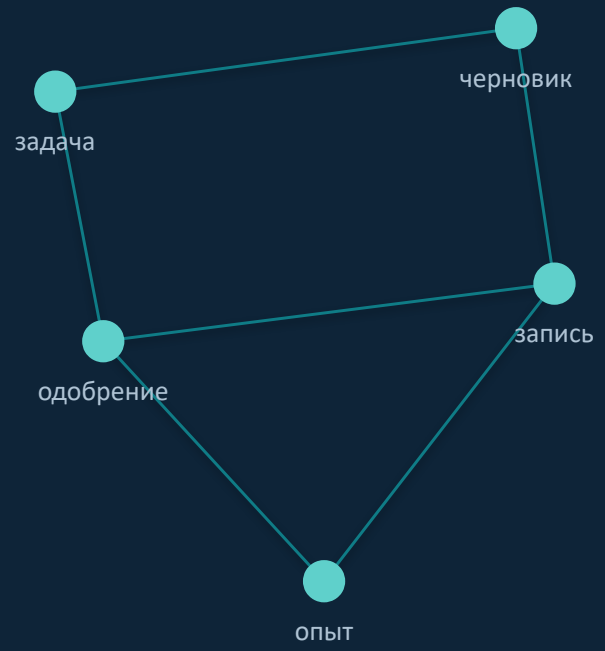
Форум, секция информатики · 14 октября 2026

Oneiro-OSTIS: автономный агент с памятью в графе знаний

Работа строится вокруг вопроса: какое наблюдение подтверждает каждое действие?



Ступчик Юлиан Владиславович, учащийся 10 класса

Государственное учреждение образования «Дубненская средняя школа имени Андрея Степановича Данилова», Гродненская область

Научный руководитель: Ковалевская Ольга Вячеславовна, учитель информатики

Гродненская область, 2026

Как устроена работа и роли



Три части системы

Модель предлагает текст и вызовы инструментов

Исполнитель проверяет право на вызов

Память удерживает записи и связи

Внешние инструменты подключаются по MCP: сервер объявляет инструмент и схему аргументов

Менеджер

ведёт разговор и ставит задачу

права на отправку нет

Исследователь

ищет и читает

права на отправку нет

Исполнитель

выполняет и получает право на отправку

роль записана в задаче

Права различаются

каждая роль видит только свои инструменты

граница объявлена в коде

Пакет изменений не вливается сам: одобрение привязано к точному вызову с точными аргументами.

Изменился адресат или текст — человек решает заново. Роль записана в самой задаче, поэтому видно, кто её выполнял.

python/oneiro/tools.py (ROLE_EFFECTS) · python/oneiro/records.py (Approval.covers) · python/oneiro/tasks.py (ROLES)

Память: типизированные записи и происхождение



Виды записей

сообщение	задача	действие	одобрение
наблюдение	опыт	заметка	курируемый факт

Больше всего мне важны решения и ошибки: по ним видно, как система работала и где ошибалась.

Запись — не строка журнала переписки: у неё есть вид, источник и связи с другими записями.

Связи

Задача связана с запуском, запуск — с действием, одобрение — с точным вызовом. По этим связям видно, откуда взялась запись и что за ней последовало.

Происхождение записи

сказал владелец
указание владельца: это утверждение человека

заявила модель
утверждение модели, а не проверенное доказательство

наблюдал инструмент
наблюдение: то, что видел вызов

проверил результат вызова
проверенный результат, а не обещание

У каждой записи есть дата создания и дата изменения.

По источнику видно, можно ли опираться на запись.

python/oneiro/records.py (Provenance) · python/oneiro/agent.py (remember → MODEL_CLAIM)

Правка не стирает прежнее значение



Владелец пишет по-русски и по-английски

источник: указание владельца · записан: 2026-10-13 01:29 · изменён: 2026-10-13 01:29

правка

Прежнее значение — остаётся в записи с датой и источником

Владелец пишет по-русски

источник: указание владельца · изменён: 2026-10-13 01:29

Что делает система

Старая редакция того же факта остаётся рядом — со своей датой и своим источником.

Панель называет, прочитана ли история записей целиком: оборванное чтение не выглядит полным.

Помечается прежняя редакция того же факта. Суждения о совместимости двух разных записей нет: обосновать его система не может, и видимость пометки была бы хуже её отсутствия.

Как это проверено: живая проверка записывает факт, правит его и читает раздел обратно — 22 шага, код 0.

Предел курируемой памяти — 2000 символов, и он проверяется до записи: если сохранённое не удаётся прочитать целиком, запись отказывается.

Что даёт память: три числа свежего замера



0,74

верных ответов с графом

память типизированными записями

0,76

верных ответов с журналом

тот же материал одним текстом

0,08

верных ответов без памяти

тот же вопрос, память отключена

LongMemEval, 50 вопросов, свежий прогон на текущем коде; верных ответов — доля клеток, которым судья вынес вердикт. Два хранилища идут вровень, а разница с памятью и без неё — сорокакратная.

Слабее всего — вопросы про предпочтения

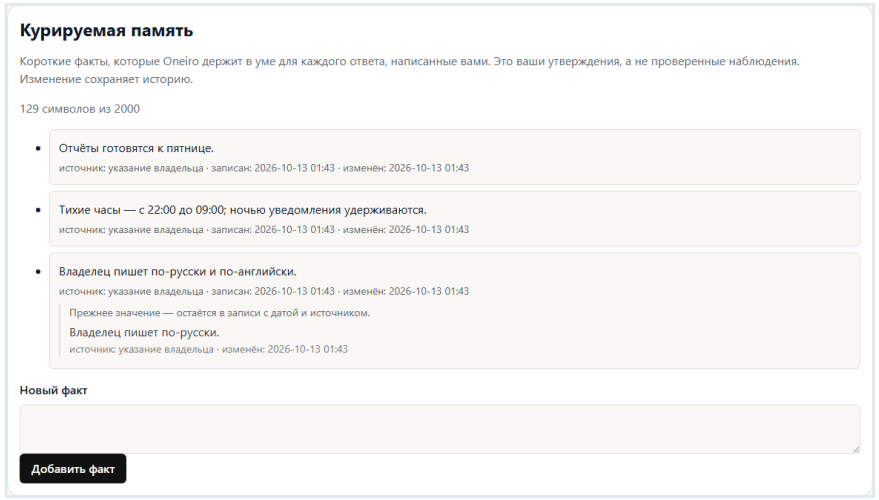
Полнота попадания 0,50 против 0,86 и 0,88 в целом; вопросы про время тоже ниже общего счёта. Это ограничение, а не победа.

Поиск по памяти — по словам и связям

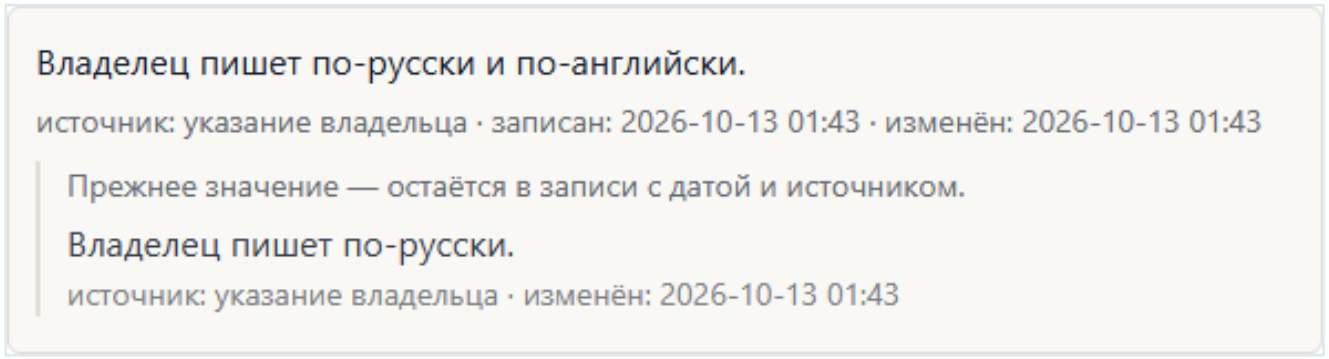
Поиск «по смыслу» — цель, а не сделанное. Одна клетка из ста пятидесяти осталась без вердикта: провайдер отказал, и она названа.

~/.openclaw/bench-memory/runs/s-50-8-report.md (свежий прогон, 50 вопросов) · bench/memory/run.py

Раздел памяти в панели



Раздел «Курируемая память» целиком: три факта владельца.



Одна запись крупно: источник, даты и прежнее значение правки.

- Строка «источник»: это утверждение владельца, а не наблюдение инструмента.
- Блок «Прежнее значение»: то, что правка заменила, со своим источником.
- Даты «записан» и «изменён»: когда запись появилась и менялась ли после.
- Счётчик символов: у курируемой памяти объявлен предел 2000.

Автономность: границы записаны в коде



1

Пакеты не вливаются сами

Изменение проходит шлюз; применение и откат — отдельные записанные действия.

2

Одобрение привязано к точным аргументам

Человек разрешает не намерение, а конкретный вызов.

3

Правила и записи меняет человек

Агент не трогает свои данные — правила и записи меняет человек.

4

Инструменты MCP объявляют схему

Нет обязательного поля — отказ с причиной, а не догадка.

5

Остановить может человек и его правила

Право на эффект объявлено ролью и настройкой разрешений.

6

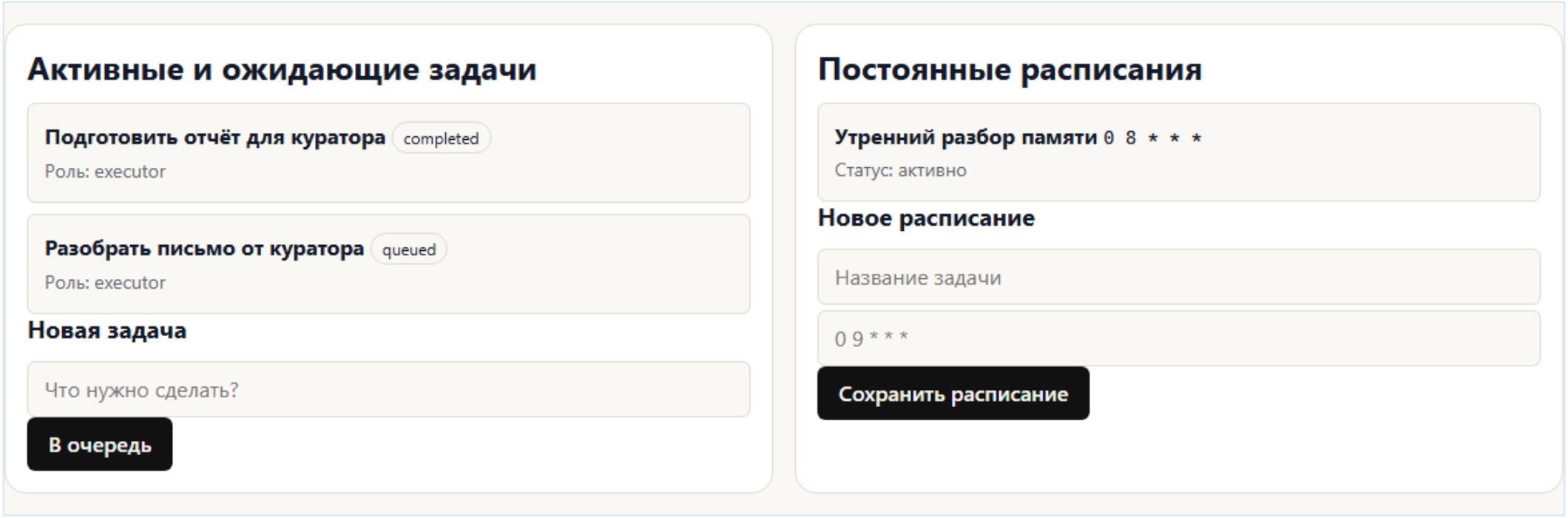
Изоляция контейнерная, не машинная

Docker и WSL2; изоляция уровня отдельной машины — цель, и она названа целью.

Система работает сама в пределах настроенных правил и останавливается там, где нужно решение человека.

python/oneiro/tools.py (arguments_digest) · python/oneiro/records.py (Approval) · python/oneiro/web.py (/api/settings)

Журнал организации



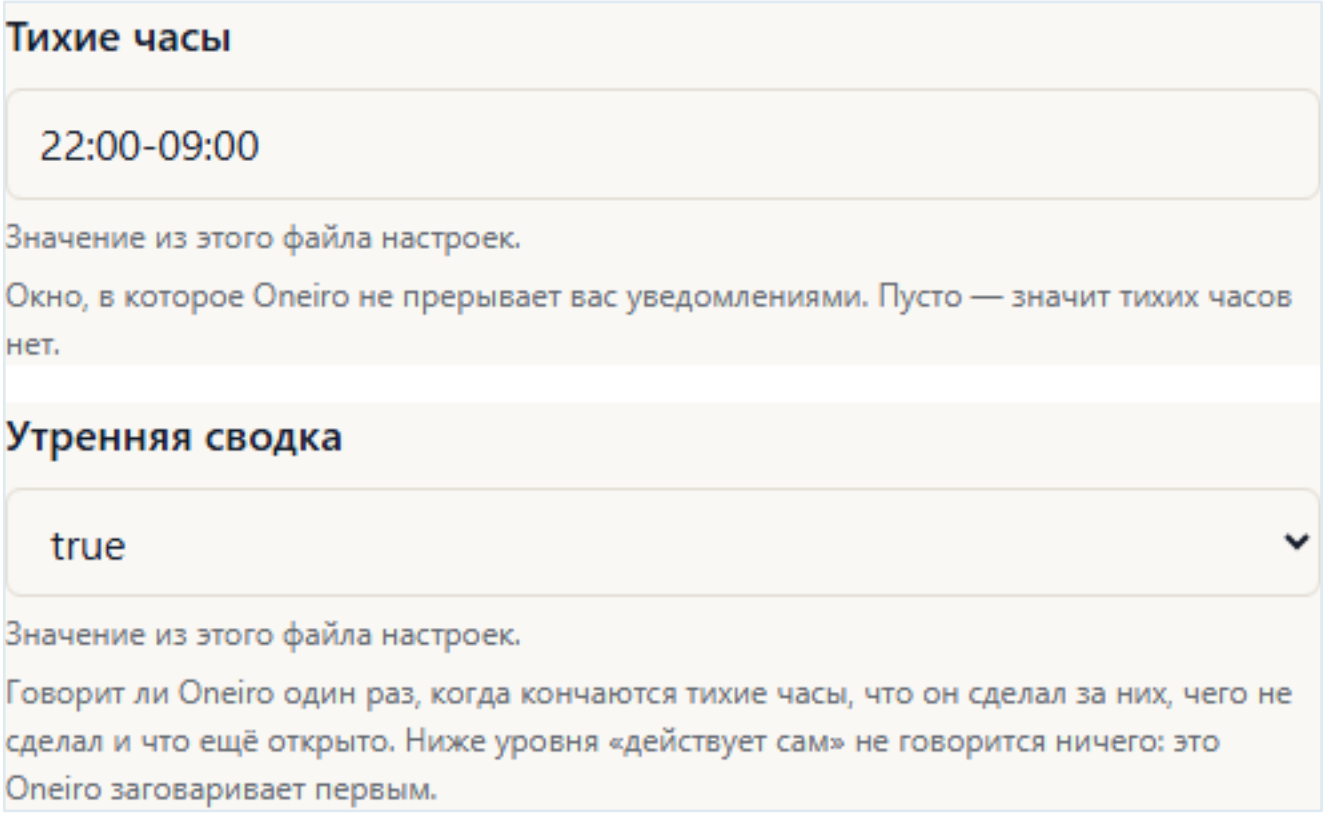
Экран задач панели: две задачи с ролями и состояниями и постоянное расписание с cron. Кадр снят tests/live_slide_shots.py.

- Роль записана в задаче, поэтому видно, кто её выполнял.
- Задача по расписанию идёт в ту же очередь и проходит те же ограничения.
- Расписание — запись в памяти, а не строка настроек: оно переживает перезапуск.
- Интервалы и расписание задаёт человек — система не выбирает их сама.

docs/forumit-2026/slides/img/ru-tasks-journal.png · python/oneiro/tasks.py · python/oneiro/schedule.py

Проактивность: расписание и тишина

- Работа в фоне по расписанию: интервалы и время задаёт человек.
- Тихие часы 22:00-09:00: работа идёт, уведомления удерживаются, а не приходят ночью по одному.
- Утром — один разбор: что завершено, что ждёт решения, где был отказ.
- Если человек не ответил, напоминание приходит один раз — не бесконечно.
- Разбор и напоминание решает тот же тик, что и расписание: второго таймера нет.
- Система не молчит до конца: она либо делает, либо говорит, что не смогла.



Фрагмент раздела «Инициатива» крупно: тихие часы со значением и объяснением и утренняя сводка. Раздел целиком — img/ru-initiative.png.

python/oneiro/proactive.py · python/oneiro/schedule.py (tick) · tests/live_proactive_check.py — 23 шага, код 0

Самоулучшение: механизм отказа



1

Повтор только по записи

Эпизод повторяется там, где решение уже наблюдалось. Где решения нет, эпизод обрывается, а покрытие показывается числом.

2

Три повтора на задачу

Кандидатов сравнивают на настоящих запусках: одиночный запуск легко попадает на хорошую сторону случайности.

3

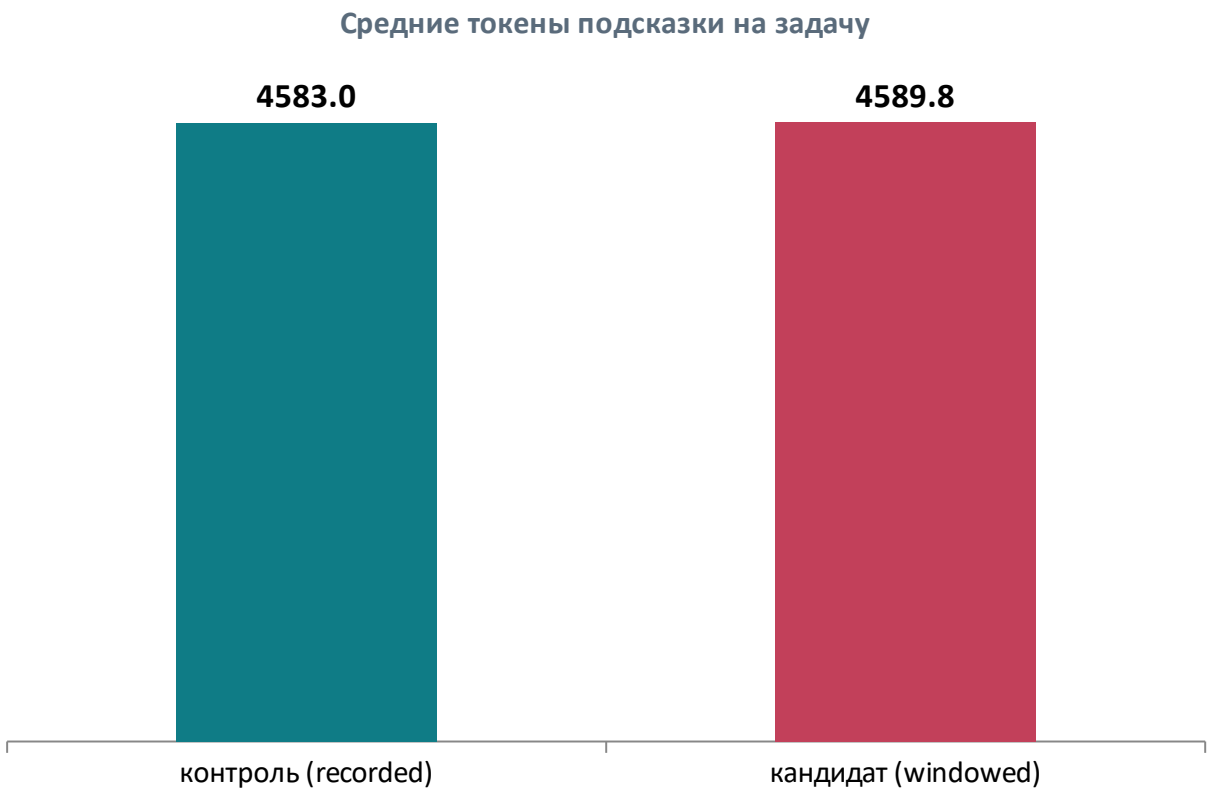
Шлюз, применение, откат

Новое поведение не применяется само. Применение и откат — отдельные записанные действия, утверждает изменение человек.

Масштаб механизма измерен: семь контрольных политик, написанных вручную, оценили по записям и измерили на запусках.

Оценка судьи разошлась с замером от −43 % до +26 % — поэтому кандидатов сравнивают запусками, а не оценкой.

Отклонённый кандидат: одно число



4589,8 против 4583,0

на 0,15 % дороже контроля

Разница не вышла за шум: beyond_noise — false.

Кандидат отклонён. Это и есть полезный результат: система отказалась от изменения, которое выглядело разумным, но не подтвердилось.

Ускорения и экономии не обещаю: в замере их нет.

docs/O8-2026-10-12.md, строки 119-130 и 241 · payload _tmp/o8-live6-payload.json

Что дальше



Поиск по смыслу по памяти

Сегодня поиск токенный: по словам и связям. Семантический поиск — следующая работа.

Изоляция уровня отдельной машины

Сейчас изоляция контейнерная. Границу обеспечивает Docker и WSL2.

Больше повторов в замерах

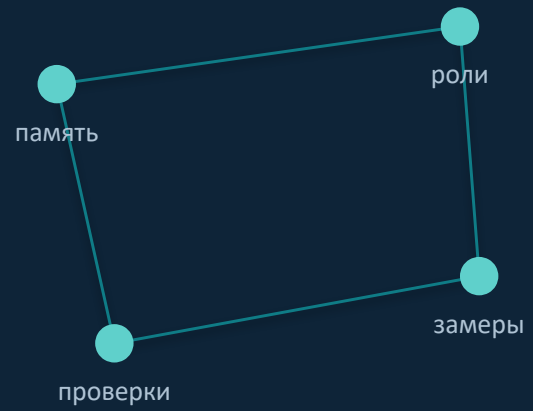
Чтобы экономия подтверждалась на нескольких наборах задач, а не на одном.

Отказ уже работает: следующий шаг — масштаб измерения.

Для меня главный вывод — не набор возможностей, а понятные границы уверенности: система различает намерение, разрешение, исполнение и подтверждение. По этому различию человек доверяет результату ровно настолько, насколько он проверен.

Ссылка на исходный код

github.com/youlianvr/oneiro-ostis



Прогон показа — scripts/forum-demo.py

Живые проверки — tests/live_*.py: вебхук, черновики, память, проактивность, офисная среда

Речь, план и доказательства — docs/forumit-2026/

Спасибо.

Готов ответить на вопросы: что показано, чего ещё нет и почему.

Репозиторий открыт: проверено 2026-10-13 (api.github.com/repos/youlianvr/oneiro-ostis → HTTP 200)